

COMP1009

Lecture 3 – An Introduction to Object Oriented Design

Lectures so far

- Overview of what objects and classes are
- Converting C code into Java
 - Creating your first class
 - Using a class (StringBuffer)
 - Using Strings – String in Java replaces C char*
 - Object references act like pointers
 - Command line arguments
- Do lab 2 to fully understand Lecture 2
 - And note the quick reference guide I produced

Summary: Java vs C/C++ differences

- Java tries to promote true object oriented programming
 - Removed global functions (can use static members though)
 - Library functions often take or return temporary objects
 - Speed and efficiency is not the **primary** concern
- **No pointers! – simplifies a lot of things and removes potential bugs**
- No malloc() and no sizeof() is needed, uses 'new' operator instead
 - Deletion of objects is done for you
- **Strings are dealt with as String or StringBuffer/StringBuilder classes**
- No objects on the stack (including arrays – **see next slide**)
 - **Objects only on heap** (created via 'new', not the stack) – always use new
 - Object references (i.e. pointers to objects) can be local variables though
- No access directly to memory locations
 - Issues for device drivers or CUDA, no optimisation of data layout/caching
- Some syntactic differences between C++ and Java
 - No use of * to dereference pointers
 - Uses . rather than -> (has no objects on stack, so no other use for .)
 - No ; at the end of class definitions (structs in C have a ; at the end)

We don't have C-type 'stack objects'

Assume struct: `struct demo { int v; };`

- Stack objects: do not exist in Java, fast but risky in C

```
struct demo d; // Object on stack : not in Java
```

```
d.v = 1;
```

```
printf("%d\n", d.v);
```

- Heap objects: exist in Java but use new & no free()

```
struct demo* pd;
```

```
pd = malloc(sizeof(struct demo)); // Use new here
```

```
pd->v = 2; // Java uses .
```

```
printf("%d\n", pd->v);
```

```
free(pd);
```

Translations C -> Java summary

- Global functions Static method
(member function)
- Arrays *Format change:* [] after type
Bonus: we know length!
- Pointers don't exist Object references do though
- char* C-type string String class for strings
StringBuffer or StringBuilder
if *mutable*
- Mutable means changeable, e.g. you can change the characters in a StringBuffer but not a String

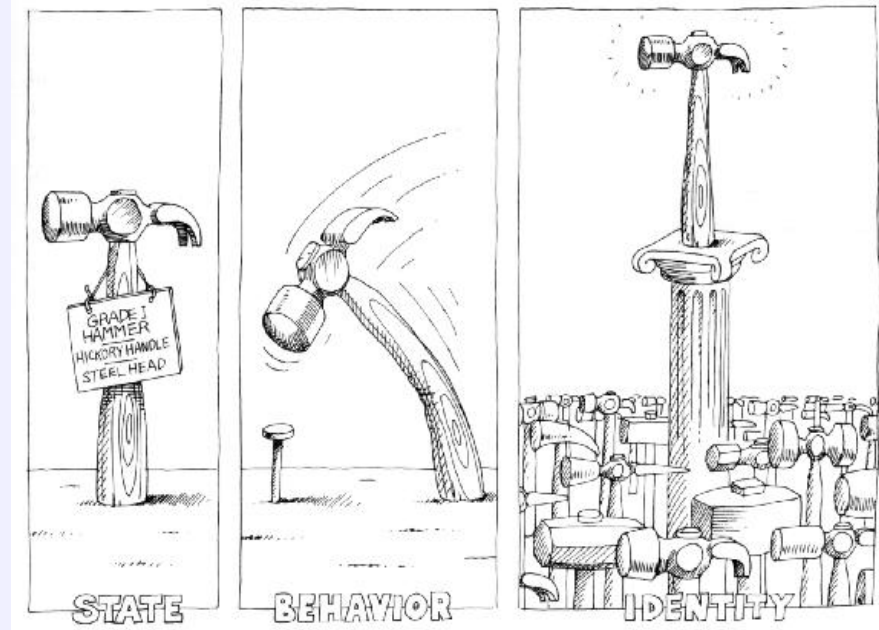
This Lecture

- Object Oriented Design
- What are objects and classes
 - We used StringBuffer
 - I did *object.function()* – what did this mean?
- Identifying objects from a system description
- Using class libraries

Object Oriented Programming

Object Orientation

- OO Systems are viewed as compositions of **domain specific object abstractions**, rather than global data and functions
 - i.e. this class/object represents this 'thing' in my program (domain)
- **An Object is a domain concept that has :**
 - **State** – data values in the object
 - **Behaviour** – methods on it
 - **Identity** – unique id (e.g. address)
- Objects intercommunicate by 'sending messages' to each other (OO terminology)
- Actually: a function on one object either calls a function on another object (synchronous call, call you)
- Or: it sets a value for another object to pick up (asynchronous, 'leave a message', text you)



A simple class – with data & methods

```
public class MyClass
{
    protected String myText;

    public void setMyText(String str)
    {
        myText = str;
    }

    public String getMyText()
    {
        return myText;
    }
}
```

A simple class – labelled

```
public class MyClass  
{
```

Data, like struct members, hidden inside class
Declared inside class, NOT inside a function

```
    protected String myText; // or private
```

```
    public void setMyText(String str)
```

```
    {  
        myText = str;  
    }
```

‘Setter’ : A method which
allows us to set the value of
the data in the object

‘Mutator’

```
    public String getMyText()
```

```
    {  
        return myText;  
    }
```

‘Getter’ : A method which
allows us to get the current
value of the data in the object

‘Accessor’

Note: None of these
functions are ‘static’
So, each is associated
with a specific object

```
}
```

Creating and using an object

```
public class MyMain
{
    public static void main(String[] args)
    {
        MyClass object = new MyClass();

        object.setMyText("Test");

        System.out.println(object.getMyText());
    }
}
```

Creating and using an object – labelled

```
public class MyMain  
{
```

main(), a static method

```
    public static void main(String[] args)  
    {
```

```
        MyClass object = new MyClass();
```

Create a new object of this type

```
        object.setMyText("Test");
```

Use the methods on the object

```
        System.out.println(object.getMyText());
```

```
    }  
}
```

Decomposition – Why?

- Commonly thought that short term memory can cope with 7 +/- 2 things at once
 - Decomposition reduces how much you have to consider at the same time
- E.g. Procedural decomposition splits the program into a number of procedures/functions, grouping operations
- Having fewer, interacting, 'things' makes the problem simpler to keep in mind
 - E.g. call a few functions, which call other functions, etc
- Object oriented design involves modelling the 'world' in objects (or classes) rather than functions
- **The problem is to identify appropriate objects**
 - **There is not usually a unique correct answer**

Thinking of objects...

- Assume we need to create a program which will:
 - Load some text from a file into memory
 - For each line, replace all occurrences of 'a' by 'b'
 - Save the text as a new file
- We want to think of this program as a collection of object/things. Consider:
 - What **types of objects** (classes) are there?
 - What can the **objects do** (be done with the objects)?
- **Q: How could we start out on this task?**
 - **Any ideas of how we could identify objects?**

Q: identify potential objects

- Assume we need to create a program which will:
- Load some text from a file
- For each line,
 replace all occurrences of 'a' by 'b'
- Save the text
 as a new file

Identifying possible objects...

- Assume we need to create a **program** which will:
- Load some **text** from a **file**
- For each **line**,
 replace all occurrences of 'a' by 'b'
- Save the **text**
 as a new **file**

Classes and objects : (often) nouns

- We need to create a **program** which will:
 - Load some **text** from a **file**
 - For each **line**, replace all occurrences of 'a' by 'b'
 - Save the **text** as a new **file**
- Program One instance, contains main()
- File Loaded file, saved file
- Text File contents
- Line Line of the file content

My example classes

Program

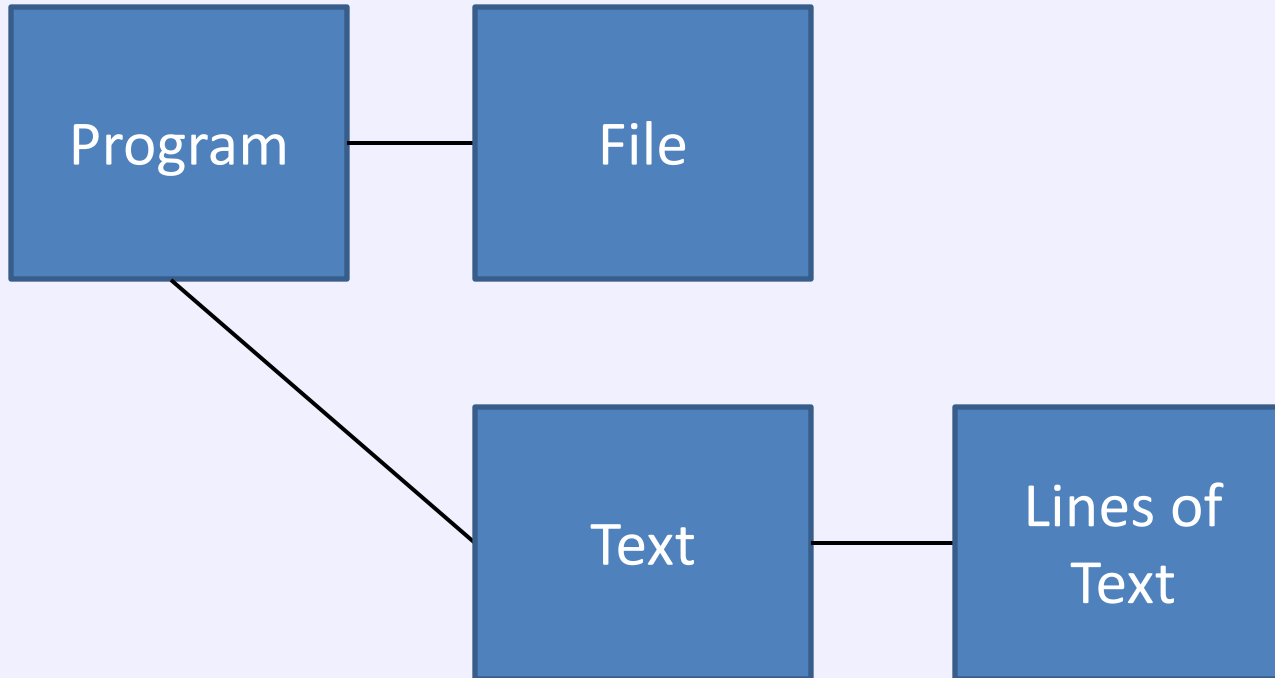
File

Text

Lines of
Text

- Are there any links between any of these?
- E.g. do any classes use other classes?

Some are linked...

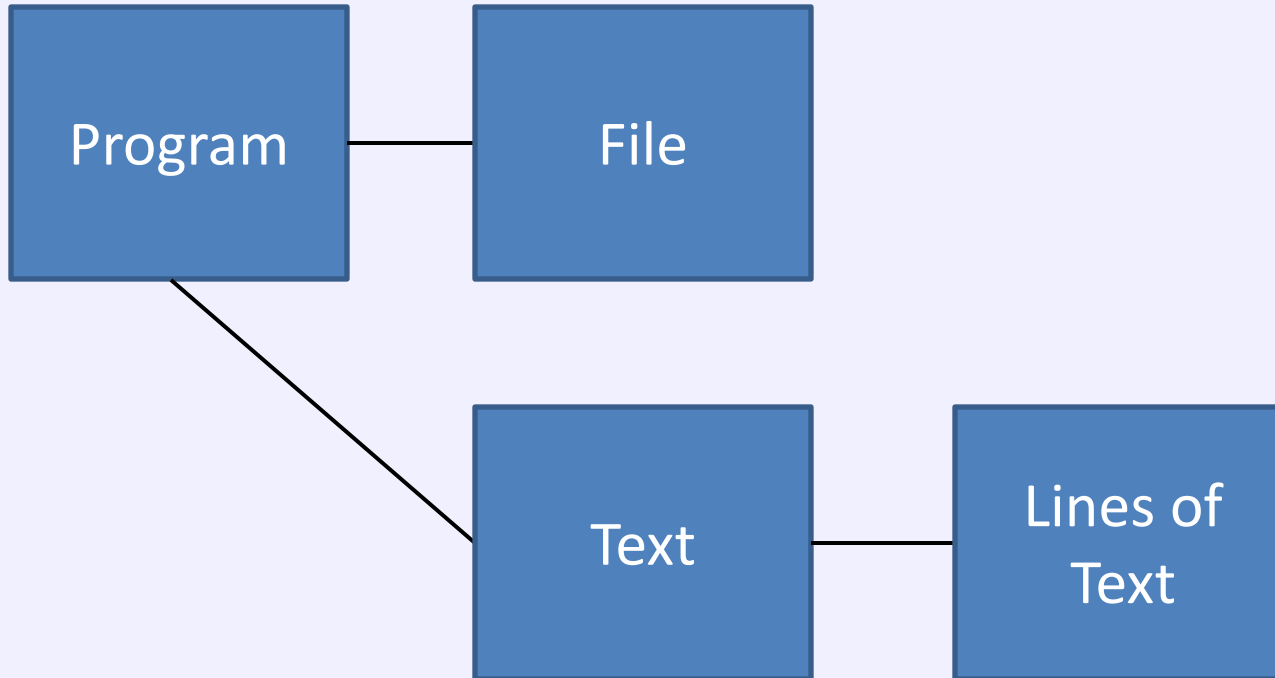


- Classes:
 - File: 2 files? 1 for reading, 1 for writing?
 - Text: The content of the file – a collection of lines
 - Lines of text: a single String / StringBuffer()
- And there are some links between these objects

Aside: ER diagrams for databases

- You will see/have seen ER diagrams in databases
- There are some similarities
 - Where we are thinking about the objects storing data, they are a lot like entities (tables)
 - And the attributes they have are like attributes/data
- As we move on, we will see that we use objects for a lot of other things as well as data storage
- Actually, the **operations/functions/things that they do will be more important to us than the data**
 - Many objects have no data!
 - Objects could exist just to collect functions together
- Relationships are also more complex in OO

What can my classes do?



- Think about what you may want these classes to be able to do?
 - And what 'data' do they need to contain
- Or what might you want to do with these classes?

Q: Identify the operations/tasks

- **What are the actions/tasks/operations (and what does each do)?**
- **Load some text from a file**
- **For each line,**
replace all occurrences of 'a' by 'b'
- **Save the text**
as a new file
- You could think of how the C code would work if that makes it easier, then assign tasks to objects

Classes:

Program

File access object

Text container

Lines of text

Java has a lot of ready-made classes

- Java classes exist for most of what you would want to do
- We use them by creating instances (objects)
 - And then calling methods on the objects
- Using the standard classes directly means that I would need to explain too many things, so we will delay that...
- I've **wrapped up** the **file input** and **file output** standard Java classes for you – we will see how in lec 4, 5 & 6
 - Class 1: **PGPFile** : file open, read, write, close
- I've also **wrapped up** a text container to contain Strings
 - Class 2: **PGPText** : store text lines, retrieve them
- I've created a **PGPInput** class that you can try in exercises
- **Abstraction** means that we can deal with the classes on the basis of 'what they do' not 'how they do it'
 - i.e. you can ignore 'how they do it' for the moment

Abstraction of PGPFile

```
public class PGPFile
```

```
{
```

```
    // Open a file for reading – fopen( filename, “r”)  
    public boolean openReadFile( String strFileName )  
    // Read the next line from a file which is open- fgets
```

```
    public String readNextLine()
```

```
    // Close a file which was open for reading – fclose()  
    public boolean closeReadFile()
```

```
    // Open a new file for writing to - fopen( filename, “w” )
```

```
    public boolean openWriteFile( String strFileName )
```

```
    // Write a line to the file - fputs/fprintf(“%s”,...)
```

```
    public void writeLine( String strLine )
```

```
    // Close a file which is open for writing – fclose()
```

```
    public boolean closeWriteFile()
```

```
}
```

I created this new (very simple) class to work a bit like the way that the C functions use files

Hopefully this simplifies usage for you

We will look at the code in a few lectures' time

Compare these (Java vs C)

```
PGPFile file = new PGPFile();  
file.openReadFile("MyTestFile.txt");  
String strLine = file.readLine();  
file.closeReadFile();
```

Java

```
char data[256];  
FILE* pFile = fopen("MyTestFile.txt", "r");  
fgets( data, 256, pFile );  
fclose(pFile);
```

C

Compare these : note the use of .

Java

```
PGPFile file = new PGPFile();  
file.openReadFile("MyTestFile.txt");  
String strLine = file.readLine();  
file.closeReadFile();
```

We act on an object. Use syntax:
objectReference . methodName()

```
char data[256];  
FILE* pFile = fopen("MyTestFile.txt", "r");  
fgets( data, 256, pFile );  
fclose(pFile);
```

C

Abstraction of PGPText

```
public class PGPText
{
// Constructor
public PGPText()
// Add a new line to the text storage
public void addLine( String strText )
// Retrieve a specific line from the store
public String getLine( int iLine )
// Store a specific line in the store
public void setLine( int iLine, String strText )
// Get a count of the number of lines in the store
public int getLineCount()
}
```

Think of this as a big
array of Strings

The array can grow
automatically whenever
it needs to

I created this too
It's really simple
We will look at it in
lecture 4

Reading the file contents

- We can use these classes in our main function

```
// Create an object to hold the lines
```

```
PGPText text = new PGPText();
```

```
// Create an object to deal with the files
```

```
PGPFile file = new PGPFile();
```

```
file.openReadFile("MyTestFileIn.txt");
```

```
String strLine;
```

```
while ( (strLine = file.readNextLine()) != null )
```

```
{
```

```
    text.addLine(strLine);
```

```
}
```

```
file.closeReadFile();
```

Example, writing the file contents

- We can use these classes in our main function

```
PGPText text = new PGPText();  
PGPFile file = new PGPFile();
```

```
int i = 0;
```

```
// (contents of next slide goes here)
```

```
file.openWriteFile("MyTestFileOut.txt");  
for ( i = 0 ; i < text.getLineCount(); i++ )  
{  
    file.writeLine(text.getLine(i));  
}  
file.closeWriteFile();
```

Example, modifying the Strings

- We can use these classes in our main function

```
PGPText text = new PGPText();  
PGPFile file = new PGPFile();
```

```
for ( i = 0 ; i < text.getLineCount(); i++ )  
{  
    String str = text.getLine(i);  
    str = str.replace('a', 'b');  
    text.setLine(i, str);  
}
```

Summary

- You can decompose tasks into interacting objects
- You can add functions/methods to classes
 - You run the method on the object using:
`objectRef.methodName()`
- These methods refer to the data for the current object
 - i.e. they know which object to work on
- Simple methods can get and set data
- More complex methods could do more, such as read from files and update the internal data appropriately

Next Lecture

- **Lab Thursday 4-5pm**
- **Do things in this order**
 - **Do Lab 1 – it gets you started with an installed IDE**
 - **Then CW2a – upload your MainFile.java**
 - **Then start on lab 2 – based on lecture 2**
- **Friday this week is live coding using PGPText etc**
 - No slides, consolidation time, to understand what we have done so far
 - Will help with Lab 3, and eventually coursework 2b
- Looking ahead: A big reason for using Object Oriented languages is the ability to easily reuse things
- Next Tuesday: Aggregation – how I built PGPText
 - Building things from components
 - Reusing things (classes) by including them into other classes
 - E.g. a car is 4 wheels + engine + ...